

Interactive Retrieval System Demo

This notebook demonstrates the interactive retrieval system that allows users to retrieve passages from the Harry Potter book using both TF-IDF and semantic search.

Steps covered:

1. Load the preprocessed text chunks
2. Initialize both retrieval methods
3. Create an interactive interface for querying
4. Compare and analyze the results

Setup

First, let's import the necessary libraries and modules.

```
In [1]: import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
from ipywidgets import interact, widgets

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import load_chunks, process_harry_potter_text
from tfidf_retrieval import TFIDFRetriever
from semantic_search import SemanticSearcher
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

1. Load the Preprocessed Text Chunks

We'll load the text chunks that were created in the first notebook. If they don't exist, we'll process the Harry Potter text.

```
In [2]: # Check if chunks file exists
chunks_path = '../data/text_chunks.json'
if not os.path.exists(chunks_path):
    print("Processing Harry Potter text...")
    chunks = process_harry_potter_text(chunks_output_path=chunks_path)
else:
    print("Loading existing text chunks...")
```

```
chunks = load_chunks(chunks_path)

print(f"Loaded {len(chunks)} text chunks")
```

Loading existing text chunks...
Loaded 972 text chunks

2. Initialize Both Retrieval Methods

Now we'll initialize both the TF-IDF retriever and the semantic searcher.

```
In [3]: # Initialize TF-IDF retriever
print("Initializing TF-IDF retriever...")
tfidf_retriever = TFIDFRetriever(chunks)

# Initialize semantic searcher
print("\nInitializing semantic searcher (this may take a moment)...")
semantic_searcher = SemanticSearcher(chunks)
```

Initializing TF-IDF retriever...
TF-IDF matrix shape: (972, 5662)

Initializing semantic searcher (this may take a moment)...
Loading model: sentence-transformers/all-MiniLM-L6-v2
Generating embeddings for all chunks...
Embedding dimension: 384
Added 972 embeddings to Faiss index

3. Create an Interactive Interface for Querying

Now we'll create an interactive interface that allows users to enter queries and see the results from both retrieval methods.

```
In [4]: def format_result(result, index):
        """Format a search result for display."""
        text = result['text']
        if len(text) > 300:
            text = text[:300] + "..."

        html = f"""
        <div style="margin-bottom: 20px; padding: 10px; border: 1px solid #ddd;
            <h4>Result {index+1} (Score: {result['score']:.4f}, Chunk ID: {result
            <p>{text}</p>
        </div>
        """
        return html

def search_and_display(query):
    """Search using both methods and display the results side by side."""
    if not query.strip():
        return HTML("<p>Please enter a query.</p>")

    # Perform searches
```

```

tfidf_results = tfidf_retriever.search(query)
semantic_results = semantic_searcher.search(query)

# Calculate overlap
tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results]
semantic_chunk_ids = [result['chunk_id'] for result in semantic_results]
common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids))
overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

# Format results
tfidf_html = ""
for i, result in enumerate(tfidf_results):
    tfidf_html += format_result(result, i)

semantic_html = ""
for i, result in enumerate(semantic_results):
    semantic_html += format_result(result, i)

# Create HTML output
html = f"""
<h2>Query: "{query}"</h2>
<p>Overlap between methods: {len(common_chunks)} out of 5 results ({overlap_percentage}%)</p>
<p>Common chunk IDs: {common_chunks}</p>

<div style="display: flex; width: 100%;">
    <div style="flex: 1; margin-right: 10px;">
        <h3>TF-IDF Results</h3>
        {tfidf_html}
    </div>
    <div style="flex: 1; margin-left: 10px;">
        <h3>Semantic Search Results</h3>
        {semantic_html}
    </div>
</div>
"""

# Save this query and results
save_query_results(query, tfidf_results, semantic_results)

return HTML(html)

def save_query_results(query, tfidf_results, semantic_results, output_dir='.'):
    """Save the results of an interactive query."""
    os.makedirs(output_dir, exist_ok=True)

    # Create a safe filename from the query
    safe_query = "".join(c if c.isalnum() else "_" for c in query)
    safe_query = safe_query[:50] # Limit filename length

    results = {
        'query': query,
        'tfidf_results': tfidf_results,
        'semantic_results': semantic_results
    }

    output_path = os.path.join(output_dir, f"{safe_query}.json")

```

```
with open(output_path, 'w', encoding='utf-8') as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
```

Now let's create the interactive widget for entering queries.

```
In [ ]: # Create the interactive widget
query_widget = widgets.Text(
    value='',
    placeholder='Enter your query here',
    description='Query:',
    disabled=False,
    layout=widgets.Layout(width='80%')
)

# Display instructions
display(HTML("""
<div style="background-color: #f8f9fa; padding: 15px; border-radius: 5px; margin: 10px 0;">
  <h3>Interactive Passage Retrieval System</h3>
  <p>Enter a query below to retrieve passages from Harry Potter using both TF-IDF and semantic search methods.</p>
  <p>Example queries:</p>
  <ul>
    <li>Harry's first day at Hogwarts</li>
    <li>Quidditch match rules</li>
    <li>Voldemort and the Philosopher's Stone</li>
    <li>Harry's scar hurts</li>
    <li>Hagrid's magical creatures</li>
  </ul>
</div>
"""))

# Create the interactive search
interact(search_and_display, query=query_widget)
```

Interactive Passage Retrieval System

Enter a query below to retrieve passages from Harry Potter using both TF-IDF and semantic search methods.

Example queries:

- Harry's first day at Hogwarts
- Quidditch match rules
- Voldemort and the Philosopher's Stone
- Harry's scar hurts
- Hagrid's magical creatures

```
interactive(children=(Text(value='', description='Query:', layout=Layout(width='80%')), placeholder='Enter your...'))
```

```
Out[ ]: <function __main__.search_and_display(query)>
```

4. Analyze Saved Queries

Let's create a function to analyze the saved queries and their results.

```
In [6]: def analyze_saved_queries(output_dir='../results/interactive_queries'):
        """Analyze the saved queries and their results."""
        if not os.path.exists(output_dir):
            return HTML("<p>No saved queries found.</p>")

        # Get all saved query files
        query_files = [f for f in os.listdir(output_dir) if f.endswith('.json')]

        if not query_files:
            return HTML("<p>No saved queries found.</p>")

        # Load and analyze each query
        queries = []
        overlaps = []

        for file in query_files:
            with open(os.path.join(output_dir, file), 'r', encoding='utf-8') as f:
                data = json.load(f)

                query = data['query']
                tfidf_chunk_ids = [result['chunk_id'] for result in data['tfidf_results']]
                semantic_chunk_ids = [result['chunk_id'] for result in data['semantic_results']]
                common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids))
                overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

                queries.append(query)
                overlaps.append(overlap_percentage)

        # Create a bar chart of overlaps
        plt.figure(figsize=(12, 6))
        plt.bar(range(len(overlaps)), overlaps)
        plt.axhline(y=np.mean(overlaps), color='r', linestyle='--', label=f'Mean Overlap: {np.mean(overlaps):.2f}%')
        plt.xlabel('Query')
        plt.ylabel('Overlap Percentage (%)')
        plt.title('Overlap Between TF-IDF and Semantic Search Results for Interactive Queries')
        plt.xticks(range(len(overlaps)), [f'Query {i+1}' for i in range(len(overlaps))])
        plt.legend()
        plt.tight_layout()
        plt.savefig('../results/interactive_queries_overlap.png')
        plt.show()

        # Create an HTML table of queries and overlaps
        html = """
        <h3>Saved Queries Analysis</h3>
        <table style="width:100%; border-collapse: collapse;">
            <tr style="background-color: #f2f2f2;">
                <th style="padding: 8px; text-align: left; border: 1px solid #ccc;">Query
                <th style="padding: 8px; text-align: left; border: 1px solid #ccc;">Overlap Percentage (%)
            </tr>
            """
```

```

for i, (query, overlap) in enumerate(zip(queries, overlaps)):
    html += f"""
    <tr>
        <td style="padding: 8px; text-align: left; border: 1px solid #dc<
        <td style="padding: 8px; text-align: left; border: 1px solid #dc<
    </tr>
    """

    html += f"""
    <tr style="background-color: #f2f2f2;">
        <td style="padding: 8px; text-align: left; border: 1px solid #dc<
        <td style="padding: 8px; text-align: left; border: 1px solid #dc<
    </tr>
</table>
    """

    return HTML(html)

```

```

In [7]: # Analyze saved queries
analyze_saved_queries()

```

Out[7]: No saved queries found.

5. Command-Line Version

For users who prefer a command-line interface, we've also implemented a command-line version of the interactive retrieval system. This can be run from the terminal using the following command:

```
python ../src/interactive.py
```

The command-line version provides the same functionality as the interactive widget above, but in a terminal interface.

Summary

In this notebook, we've:

1. Loaded the preprocessed text chunks
2. Initialized both retrieval methods
3. Created an interactive interface for querying
4. Analyzed saved queries
5. Provided information about the command-line version

This interactive retrieval system allows users to directly compare the results from TF-IDF and semantic search, providing insights into the strengths and limitations of each approach. By experimenting with different types of queries, users can see how the two methods handle various retrieval scenarios.

